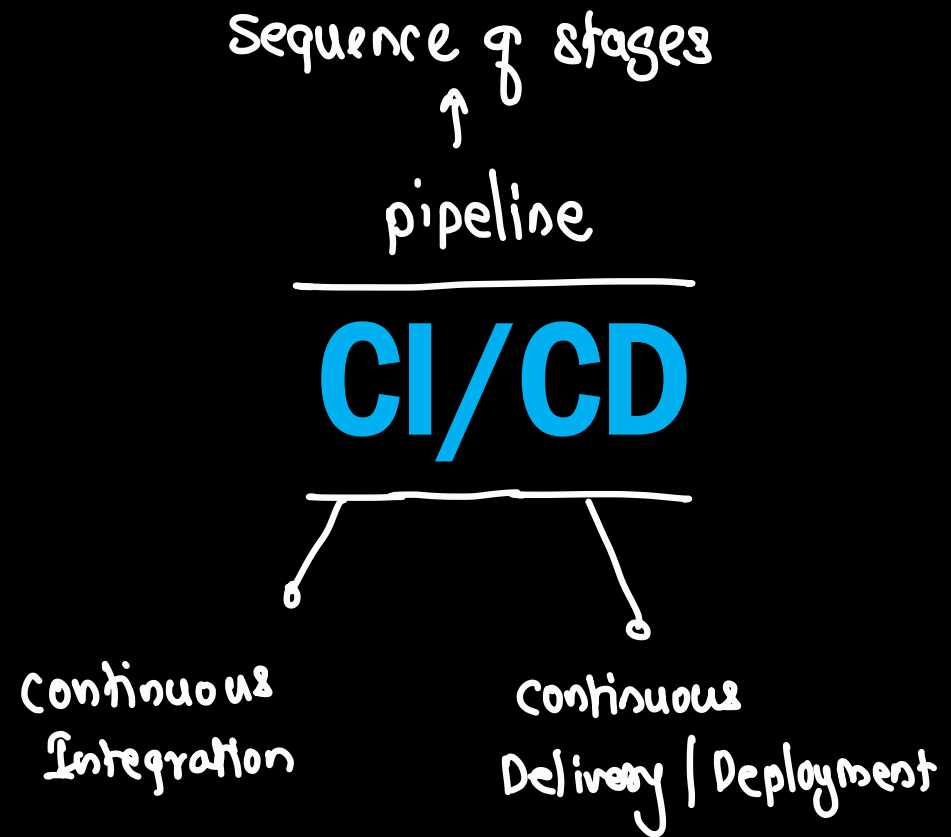




What is CI/CD Pinepines



- A CI/CD pipeline automates the steps required from writing code - delivering the application - deploying it to production

- It ensures:

- Faster development cycles
- Higher code quality
- Fewer manual errors
- Repeatable and reliable deployments

↓
accessible to end users

- Benefits of CI/CD pipeline

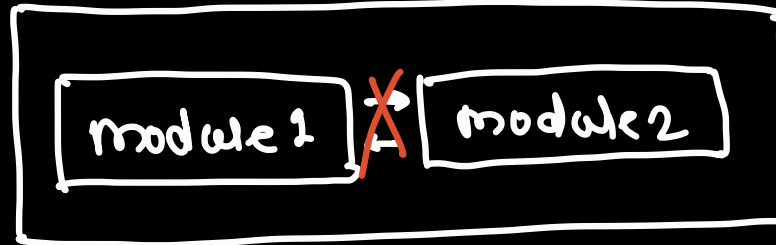
- Business Benefits

- Faster time to market
 - Less downtime
 - Lower cost of development
 - Increased reliability

- Developer Benefits

- Stable codebase
 - Automated workflows
 - Faster feedback loop
 - Easier collaboration

System - application

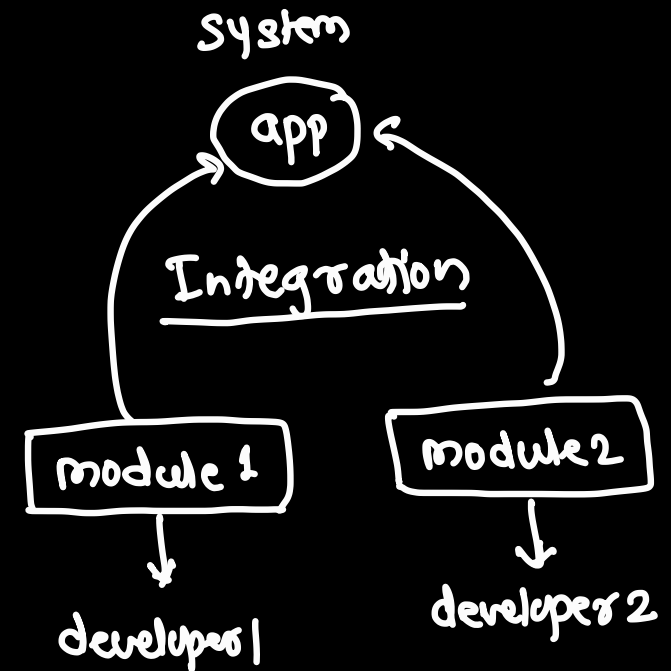


Continuous Integration

- This is the practice of automatically integrating code changes from multiple developers into a shared repository frequently—often several times a day
- What happens:
 - Developers push code to a version control system (like Git)
 - Automated tests run to check if the new code breaks anything
 - The code is automatically built/compiled
 - Teams get immediate feedback if something is wrong

build → compiling application & adding
required resources to create deployable
package

· java → compiled → .class
· xml → compiled → .arsc
· } → .apk



The Problem CI Solves



■ Before CI

- Developers worked on separate branches for weeks or months
- When merging, massive conflicts occurred ("integration hell")
- Bugs were discovered late, making them expensive to fix
- No one knew if the combined code actually worked until the end
- Building and testing were manual, slow, and error-prone

■ With CI

- Small, frequent integrations reduce conflict complexity
- Problems are detected within minutes, not weeks
- The codebase is always in a working state
- Team collaboration improves with better visibility

Core Principles of CI



- **Single Source Repository**
 - All code lives in one version control system (Git, SVN, etc.). Everyone commits to the same repository, typically using a branching strategy like GitFlow or trunk-based development.
- **Frequent Commits**
 - Developers commit code at least once per day, ideally multiple times. Smaller, incremental changes are easier to integrate and debug than large batches.
- **Automated Build**
 - Every commit triggers an automated build process that compiles the code and creates executable artifacts. This ensures the code can actually be built successfully.
- **Self-Testing Build**
 - The build includes running automated tests. If tests fail, the build is considered broken. Common test types include:
 - Unit tests - Test individual functions/components
 - Integration tests - Test how components work together
 - Static code analysis - Check code quality and standards
 - Security scans - Identify vulnerabilities
- **Fast Builds**
 - Builds should complete quickly (ideally under 10 minutes) so developers get rapid feedback. Slow builds discourage frequent commits.
- **Test in Production-Like Environment** → *pre-prod env*
 - CI environments should mirror production as closely as possible to catch environment-specific issues early.
- **Easy Access to Latest Build**
 - Everyone on the team can access the latest executable and test results. Transparency is crucial.
- **Visible Build Status**
 - The current build status is highly visible to the team through dashboards, notifications, or physical indicators (like lava lamps or traffic lights in some offices).
- **Fix Broken Builds Immediately**
 - When a build breaks, fixing it becomes the top priority. The team shouldn't commit new changes until the build is green again.

The CI Workflow

unit test cases (test individual function)

Step 1: Developer Works Locally

- Developer pulls the latest code from the main branch
- Makes changes and writes tests → TDD → test driven development
- Runs tests locally to ensure everything works

Step 2: Commit and Push

- Developer commits changes with a descriptive message
- Pushes code to the central repository

Step 3: CI Server Detects Change

- The CI server (Jenkins, GitLab CI, etc.) monitors the repository
- Detects the new commit via webhooks or polling

Step 4: Checkout Code

- CI server pulls the latest code from the repository (main branch)
- Creates a clean build environment

Step 5: Build Process

- Compiles the source code
- Resolves dependencies
- Creates build artifacts (executables, containers, packages)

Step 6: Run Tests

- Executes automated test suites
- Generates test reports and code coverage metrics
- May run multiple test stages in parallel for speed

Step 7: Code Quality Checks

- Runs linters to check coding standards
- Performs static code analysis
- Checks for security vulnerabilities
- Verifies code coverage meets thresholds

Step 8: Generate Artifacts

- If all checks pass, creates deployable artifacts
- Tags the build with version numbers
- Stores artifacts in a repository (Artifactory, Nexus, container registry)

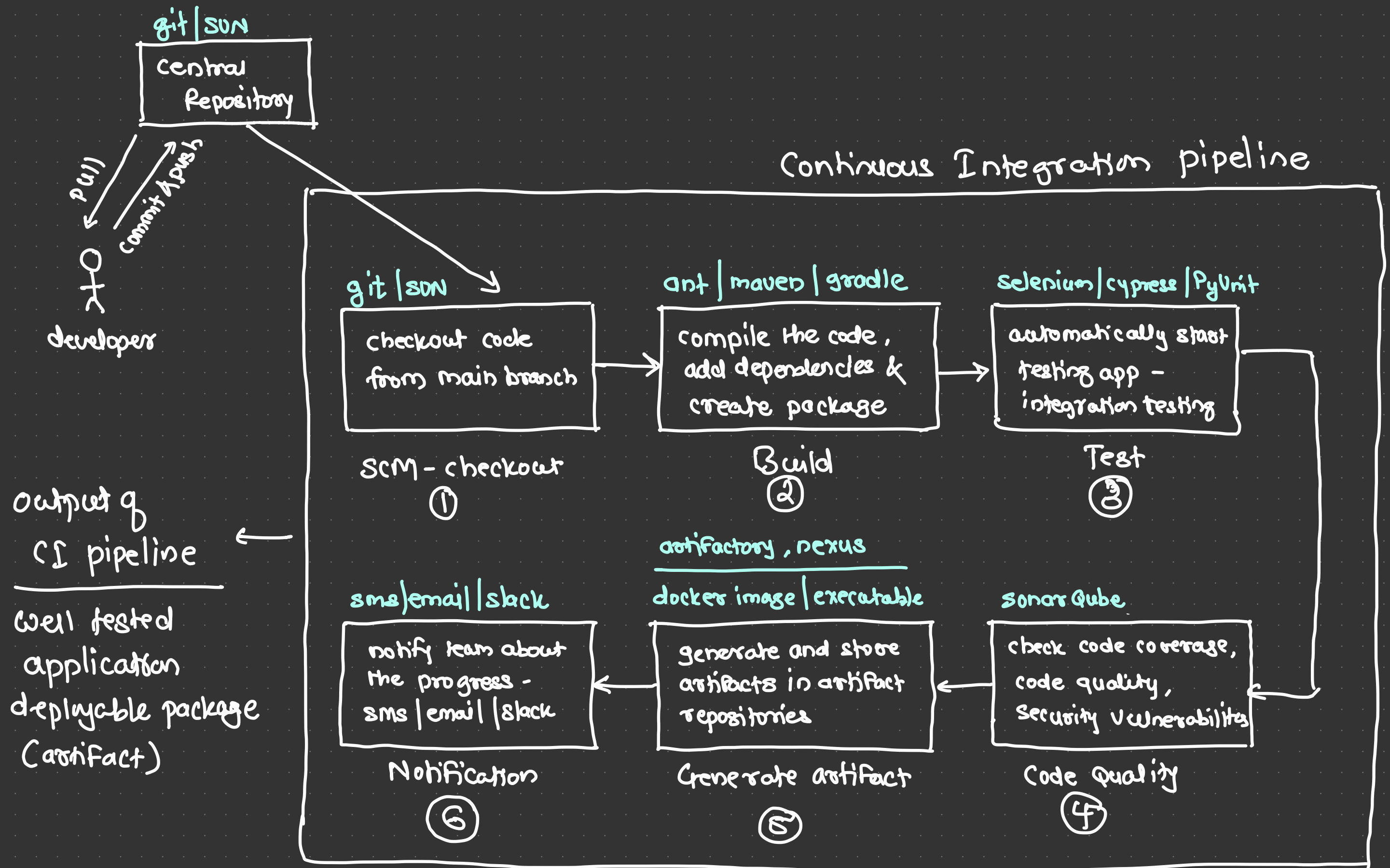
Step 9: Notification

- Sends results to the team (email, Slack, dashboard)
- Updates build status badges
- Green = success, Red = failure

Step 10: Feedback Loop

- If successful: Developer continues with next task
- If failed: Developer investigates logs, fixes issues, and recommits

main/master branch → has latest, well tested, stable, crash free code.



Key CI Components



■ Version Control System (VCS)

- Git (GitHub, GitLab, Bitbucket) → distributed VCS
- Subversion → centralized VCS
- Mercurial →

■ CI Server/Platform

- Jenkins - Most popular, highly customizable, open-source
- GitLab CI/CD - Integrated with GitLab, uses YAML configs
- GitHub Actions - Native to GitHub, marketplace of actions
- CircleCI - Cloud-based, fast Docker support
- Travis CI - Popular for open-source projects
- TeamCity - By JetBrains, powerful and user-friendly
- Azure DevOps - Microsoft's comprehensive platform
- Bamboo - By Atlassian, integrates with Jira

■ Build Tools

- Maven, Gradle (Java)
- npm, Yarn, Webpack (JavaScript)
- MSBuild (.NET)
- Make, CMake (C/C++)
- pip, Poetry (Python)

■ Build Tools

- Maven, Gradle (Java)
- npm, Yarn, Webpack (JavaScript)
- MSBuild (.NET)
- Make, CMake (C/C++)
- pip, Poetry (Python)

■ Testing Frameworks

- JUnit, TestNG (Java)
- Jest, Mocha, Cypress (JavaScript)
- pytest, unittest (Python)
- NUnit, xUnit (.NET)

■ Code Quality Tools

- SonarQube - Comprehensive code analysis
- ESLint, Pylint - Linters for specific languages
- CodeClimate, Codacy - Cloud-based analysis
- OWASP Dependency Check - Security scanning



Benefits of CI

- **Reduced Integration Risk** - Small, frequent merges prevent "integration hell"
- **Higher Code Quality** - Automated testing catches bugs early
- **Faster Feedback** - Developers know within minutes if their code works
- **Improved Collaboration** - Visibility into what everyone is working on
- **Reduced Manual Testing** - Automation frees up time for exploratory testing
- **Documentation** - Build logs provide a history of changes and issues
- **Confidence** - Teams can refactor and improve code without fear
- **Faster Time to Market** - Streamlined process accelerates delivery

Continuous Delivery and Continuous Deployment



- CD refers to two related but distinct practices that extend Continuous Integration:
- **Continuous Delivery** - Code changes are automatically built, tested, and prepared for release to production, but deployment requires manual approval.
- **Continuous Deployment** - Every change that passes all automated tests is automatically released to production without human intervention. → *automatic deployment*
- Both ensure that code is always in a deployable state, but they differ in the final step to production.

The Problem CD Solves



■ Before CD

- Releases happened infrequently (monthly, quarterly, or even yearly)
- Manual deployment processes were error-prone and stressful
- Large releases contained many changes, making issues hard to trace
- Deployment knowledge was siloed with specific team members
- Testing in production-like environments happened too late
- Rollbacks were complex and risky ✖
- Long lead times from code completion to customer value

■ With CD

- Releases can happen multiple times per day
- Deployments are automated, consistent, and repeatable
- Small, incremental changes reduce risk
- Anyone on the team can deploy with a button click
- Issues are detected early through extensive automated testing
- Rollbacks are quick and automated
- Features reach customers rapidly

Continuous Delivery vs Continuous Deployment



■ Continuous Delivery

- Manual gate before production deployment
- Humans decide when to release
- Business decides the release schedule
- Suitable when releases need coordination (marketing, training, etc.)
- Common in enterprise, regulated industries, or when deploying to client-managed environments
- Example Flow: Code → Build → Test → Stage → [Manual Approval] → Production

■ Continuous Deployment

- Fully automated pipeline to production
- Every successful build goes live automatically
- No human intervention in the deployment process
- Requires mature testing and monitoring
- Common in SaaS products, web applications, and companies like Netflix, Amazon, Etsy
- Example Flow: Code → Build → Test → Stage → Production (all automated)

Core Principles of CD



- **Build Quality In**
 - Quality isn't checked at the end; it's built into every step. Automated testing at multiple levels ensures confidence in releases.
- **Work in Small Batches**
 - Deploy small, incremental changes frequently rather than large batches. Smaller changes are easier to test, deploy, and troubleshoot.
- **Automate Everything**
 - From building to testing to deployment, minimize manual steps. Automation ensures consistency and reduces human error.
- **Relentless Pursuit of Continuous Improvement**
 - Continuously optimize the pipeline, reduce cycle time, and improve feedback loops.
- **Everyone is Responsible**
 - The entire team owns the deployment pipeline, not just operations. Developers are accountable for production readiness.
- **Done Means Released**
 - Work isn't complete when code is written—it's complete when it's delivering value in production.

The CD Pipeline Stages



■ Stage 1: Source Control

- Everything starts with code committed to version control. This triggers the entire pipeline.

■ Stage 2: Build Automation

- Compile source code
- Resolve and download dependencies
- Create build artifacts (JAR files, Docker images, executables)
- Version and tag artifacts
- Store in artifact repository
- Tools: Maven, Gradle, npm, Docker, Artifactory, Nexus

■ Stage 3: Automated Testing

- This is the most critical part of CD. Multiple layers of testing provide confidence:
- Unit Tests
- Integration Tests
- Contract Tests
- End-to-End Tests
- Performance Tests
- Security Tests

The CD Pipeline Stages



■ Stage 4: Staging/Pre-Production Deployment

- Code is deployed to an environment that mirrors production:
 - Same infrastructure configuration
 - Similar data volumes (often anonymized production data)
 - Same software versions
 - Equivalent network topology
- Activities:
 - Deploy using the same process as production
 - Run full test suites
 - Perform exploratory testing
 - Validate configuration
 - Check monitoring and logging

■ Stage 5: Production Deployment ✖✖

- The final deployment to live environment where users access the application
- Deployment Strategies:
 - Blue-Green Deployment
 - Canary Deployment
 - Rolling Deployment ✖✖

Benefits of CD



- **Faster Time to Market** - Features reach customers quickly
- **Reduced Risk** - Small, frequent deployments are less risky
- **Higher Quality** - Extensive automated testing catches issues
- **Lower Stress** - Deployments become routine, not events
- **Better Feedback** - Quick feedback from real users
- **Improved Developer Productivity** - Less time spent on manual processes
- **Competitive Advantage** - Respond to market changes rapidly
- **Lower Costs** - Automation reduces manual effort
- **Better Security** - Patches and fixes deployed quickly
- **Improved Reliability** - Consistent, tested deployment processes

Key CD Technologies and Tools



■ Infrastructure as Code (IaC)

- Define infrastructure in version-controlled files
- Terraform - Multi-cloud infrastructure provisioning
- CloudFormation - AWS-specific IaC
- Ansible - Configuration management and provisioning
- Chef, Puppet - Configuration management
- Pulumi - IaC using programming languages

■ Container Orchestration

- Manage containerized applications
- Kubernetes - Industry-standard orchestration
- Docker Swarm - Simpler alternative to K8s
- Amazon ECS/EKS - AWS container services
- Azure AKS - Azure Kubernetes Service
- Google GKE - Google Kubernetes Engine

■ Deployment Tools

- Automate the deployment process
- Helm - Kubernetes package manager
- ArgoCD - GitOps continuous delivery for K8s
- Flux - GitOps toolkit for K8s
- Spinnaker - Multi-cloud continuous delivery
- Octopus Deploy - Deployment automation
- AWS CodeDeploy - AWS deployment service

■ Configuration Management

- Manage application configuration
- Consul - Service mesh and configuration
- etcd - Distributed key-value store
- Spring Cloud Config - Centralized configuration for Spring
- AWS Systems Manager - Parameter Store and secrets
- Azure Key Vault - Secrets management

Key CD Technologies and Tools



■ Artifact Repositories

- Store build artifacts
- Docker Hub - Container images (public)
- Amazon ECR - AWS container registry
- Google Container Registry - GCP container registry
- Artifactory - Universal artifact repository
- Nexus - Repository manager
- GitHub Packages - Integrated with GitHub

■ Monitoring and Observability

- Track application health:
- Prometheus + Grafana - Metrics and visualization
- Datadog - Full-stack monitoring
- New Relic - APM and observability
- Dynatrace - AI-powered monitoring
- ELK Stack (Elasticsearch, Logstash, Kibana) - Log management
- Splunk - Log analysis and SIEM
- Jaeger, Zipkin - Distributed tracing
- Sentry - Error tracking



Jenkins

What is Jenkins ?



- Jenkins is an open-source automation server that helps software development teams automate the process of building, testing, and deploying their applications
- Think of Jenkins as a reliable assistant that can perform repetitive tasks automatically, saving developers time and reducing human errors
- It was originally called Hudson and was created in 2004 by Kohsuke Kawaguchi while he worked at Sun Microsystems
- In 2011, it was renamed to Jenkins after a dispute with Oracle, and it has grown to become the most popular automation server in the world
- Jenkins is free to use and has a large community of contributors who continuously improve it and create extensions for it

Why Jenkins



■ Strengths

- **Free and Open Source** - No licensing costs
- **Highly Extensible** - Over 1,800 plugins available
- **Platform Agnostic** - Runs on Windows, Linux, macOS
- **Technology Agnostic** - Works with any programming language or tool
- **Large Community** - Extensive documentation and community support
- **Self-Hosted** - Full control over your CI/CD infrastructure
- **Mature and Battle-Tested** - Used by thousands of organizations
- **Flexible** - Can be configured for simple to extremely complex workflows

■ Weaknesses

- **Steep Learning Curve** - Complex to set up and configure initially
- **Maintenance Overhead** - Requires dedicated resources for upkeep
- **UI/UX** - Interface feels dated compared to modern alternatives
- **Plugin Management** - Plugin compatibility issues can arise
- **Scaling Challenges** - Requires careful architecture for large deployments
- **Security** - Requires vigilant security configuration and updates

Architecture



- Jenkins operates using a master-agent architecture, which means it has one central controller (the master) and can have multiple workers (agents)
- The master server is the brain of Jenkins - it schedules builds, monitors agents, records results, and provides the web interface you interact with
- Agents are separate machines that actually execute the work - they compile code, run tests, and perform deployments as instructed by the master
- This architecture allows Jenkins to distribute work across multiple machines, enabling it to handle many projects simultaneously
- For small projects, you might run everything on the master, but for larger organizations, you would have dozens or hundreds of agents

Node Types



Dashboard

■ Master Server

- The master server is where Jenkins itself runs and where you configure all your automation tasks through a web browser
- It stores all the configuration data, including information about what tasks to run, when to run them, and where to run them
- The master continuously monitors your code repositories (like GitHub or GitLab) for changes and triggers builds when new code is committed
- It also maintains the build history, showing you which builds succeeded, which failed, and providing access to logs and test results
- Think of the master as the conductor of an orchestra, coordinating all the different pieces but not necessarily playing every instrument itself

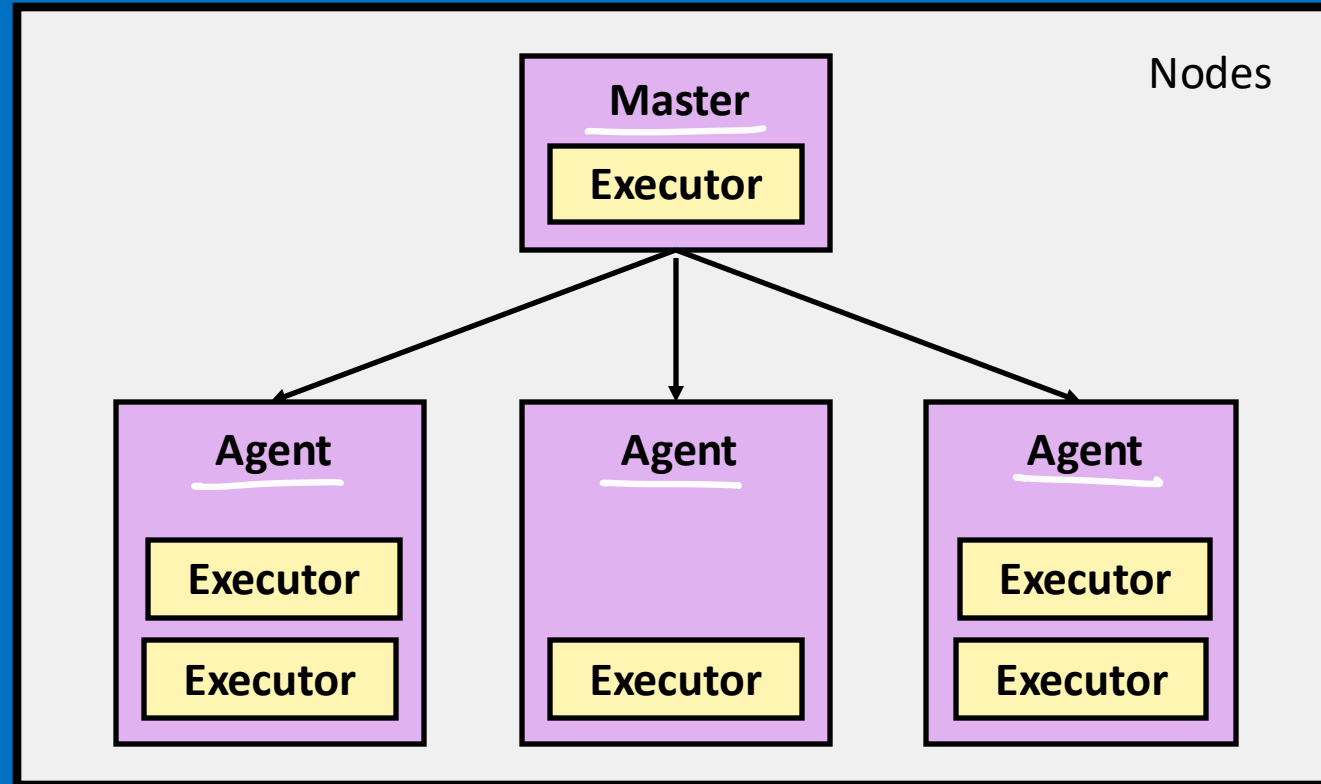
■ Agent Node

or containers

- Agents (also called slaves or nodes) are separate computers or virtual machines that connect to the Jenkins master to perform actual work
- When the master decides a build needs to run, it sends instructions to an available agent, which then executes those instructions
- Agents can run on different operating systems (Windows, Linux, Mac) allowing you to test your application on multiple platforms simultaneously
- Each agent can have labels describing its capabilities, like "linux," "docker," or "high-memory," helping Jenkins choose the right agent for each job
- Using multiple agents allows your organization to run many builds in parallel, dramatically reducing wait times when multiple developers are committing code



Jenkins Environment



Jenkins Job



- Freestyle jobs are the most basic type - they provide a simple web form where you configure each step of your automation process
- Pipeline jobs use code (written in a language called Groovy) to define your automation, making it easier to manage complex workflows and version control your automation scripts
- Multi-configuration jobs allow you to run the same job with different parameters, like testing your application on Windows, Linux, and Mac simultaneously
- Folder jobs help organize your jobs into hierarchies, similar to how you organize files on your computer into folders
- Multi-branch pipeline jobs automatically create a separate pipeline for each branch in your source code repository, perfect for teams working on multiple features simultaneously

Jenkins Builds



- A build is a single execution of a Jenkins job - every time a job runs, it creates a new build with an incrementing number
- E.g., if you have a job called "Build My App," its first execution is Build 1, the second execution is Build 2, and so on
- Each build has a status: Success (everything worked), Failure (something went wrong), Unstable (it ran but tests failed), or Aborted (someone stopped it)
- Jenkins keeps a history of all builds, allowing you to see trends over time, like whether your project is getting more stable or less stable
- You can view logs for each build to see exactly what happened during execution, which is invaluable for troubleshooting when something goes wrong

Jenkins Workspace



- A workspace is a directory on an agent's hard drive where Jenkins stores the files needed for a particular job
- When a build starts, Jenkins typically checks out your source code from a repository into the workspace, giving the build access to your files
- The workspace persists between builds of the same job on the same agent, which can speed up subsequent builds by avoiding redundant downloads
- However, this also means that leftover files from previous builds might interfere with new builds, so Jenkins provides options to clean the workspace before each build
- Each job usually has its own workspace directory to prevent different jobs from interfering with each other's files

Jenkins Plugins



- Plugins are extensions that add new functionality to Jenkins, similar to how apps add features to your smartphone
- Jenkins ships with basic functionality, but plugins allow you to integrate with hundreds of different tools and services like GitHub, Docker, Kubernetes, and Slack
- For example, the Git plugin lets Jenkins check out code from Git repositories, while the Email plugin lets Jenkins send notification emails
- There are over 1,800 plugins available in the Jenkins plugin repository, covering everything from source control to deployment to notifications
- Plugins are maintained by the Jenkins community, and you can install them through Jenkins' web interface with just a few clicks



Installing Jenkins - System Requirements

- Before installing Jenkins, your computer or server needs Java installed because Jenkins is a Java application
- Specifically, Jenkins requires Java 11 or Java 17 (these are versions of the Java runtime environment)
- You will need at least 256 MB of RAM, but 1 GB or more is recommended for any real-world usage
- About 10 GB of hard drive space is recommended - 1 GB for the Jenkins application itself and the rest for storing build data and workspaces
- Jenkins can run on Windows, Linux, or Mac operating systems, and there are installation packages available for all major platforms
- **Ubuntu/Debian**
 - `wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -`
 - `sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'`
 - `sudo apt update`
 - `sudo apt install jenkins`
- **Start Jenkins**
 - `sudo systemctl start jenkins`
 - `sudo systemctl enable jenkins`



Pipeline

Why Pipeline



- Pipeline adds a powerful set of automation tools onto Jenkins, supporting use cases that span from simple continuous integration to comprehensive CD pipelines
- **Advantages of Pipeline**
 - **Code:** Pipelines are implemented in code and typically checked into source control, giving teams the ability to edit, review, and iterate upon their delivery pipeline.
 - **Durable:** Pipelines can survive both planned and unplanned restarts of the Jenkins controller.
 - **Pausable:** Pipelines can optionally stop and wait for human input or approval before continuing the Pipeline run.
 - **Versatile:** Pipelines support complex real-world CD requirements, including the ability to fork/join, loop, and perform work in parallel.
 - **Extensible:** The Pipeline plugin supports custom extensions to its DSL ^[1] and multiple options for integration with other plugins.

Pipeline as Code



- Pipeline as Code describes a set of features that allow Jenkins users to define pipelined job processes with code, stored and versioned in a source repository
- These features allow Jenkins to discover, manage, and run jobs for multiple source repositories and branches — eliminating the need for manual job creation and management
- A Jenkins Scripted Pipeline defines a sequence of stages and steps for continuous integration and continuous delivery (CI/CD) tasks using a Groovy-based Domain Specific Language (DSL)
- It offers a highly flexible and expressive way to define pipelines as code, typically within a Jenkinsfile stored in a source code repository

Characteristics of Scripted Pipelines



- **Groovy-based DSL**
 - Scripted Pipelines leverage the power and flexibility of the Groovy programming language, allowing for complex logic and control flow
- **Imperative programming model**
 - Unlike Declarative Pipelines, which encourage a structured, declarative approach, Scripted Pipelines follow an imperative model where the exact order of steps and their execution logic are explicitly defined
- **High flexibility**
 - The Groovy foundation provides extensive control over pipeline execution, enabling advanced scenarios, custom error handling, and dynamic behavior
- **node block**
 - Scripted Pipelines typically begin with a node block, which allocates an executor on a Jenkins agent to run the pipeline
- **Stages and Steps**
 - Pipelines are organized into stage blocks for logical grouping of related tasks (e.g., Build, Test, Deploy). Within each stage, individual steps perform specific actions like compiling code, running tests, or deploying applications
 - While stage blocks are optional in the syntax, they are highly recommended for better visualization in the Jenkins UI

Pipeline Syntax



```
pipeline {  
  agent any  
  environment {}  
  stages {  
    stage('Example') {  
      environment {}  
      steps {  
        sh 'printenv'  
      }  
    }  
  }  
}
```



GitOps

What is GitOps



- **GitOps** is a modern approach to continuous deployment and infrastructure management where Git serves as the single source of truth for declarative infrastructure and applications
- The entire system state is versioned in Git, and automated processes ensure the actual state matches the desired state defined in the repository
- It was coined by Weaveworks in 2017
- It is evolved from traditional CI/CD, Infrastructure as Code (IaC)
- It solved many problems like: Manual deployments, configuration drift, lack of audit trails
- It is mainly preferred in Cloud-native applications, Kubernetes adoption etc.

Core Principles



- **Declarative Configuration**
 - The entire system (infrastructure and applications) is described declaratively
- **Version Control as Single Source of Truth**
 - Git repository contains the complete state of the system
- **Automated Synchronization**
 - Automated agents ensure the actual state matches the desired state in Git
- **Continuous Reconciliation**
 - System continuously monitors and corrects drift from the desired state

Key Components



- **Git Repository**
 - Stores all configuration files (YAML, JSON, etc.)
- **GitOps Operator/Agent**
 - Runs inside the cluster and monitors Git for changes
- **Kubernetes/Target Platform**
 - Where applications and infrastructure run
- **CI/CD Pipeline**
 - Builds images and updates Git repositories
- **Monitoring & Alerting**
 - Tracks deployment status and system health

GitOps Workflow



- Developer commits code changes to application repository
- CI pipeline builds container image and pushes to registry
- CI pipeline updates image tag in GitOps repository
- GitOps operator detects change in Git repository
- Operator pulls new configuration from Git
- Operator applies changes to Kubernetes cluster
- System reconciles to match desired state
- Monitoring confirms successful deployment

GitOps vs Traditional CI/CD



Aspect	Traditional CI/CD	GitOps
Deployment method	Push-based	Pull-based
Access Control	CI system has cluster credentials	No external access is needed
Security	Credentials exposed outside the cluster	Credentials stay within cluster
Rollback	Manual or Complex	Simple Git revert
Audit Trail	Limited	Complete Git History
Drift Detection	Manual	Automatic Reconciliation



Popular GitOps Tools

- **ArgoCD:** Most popular, rich UI, multi-tenancy support
- **Flux CD:** CNCF graduated, lightweight, GitOps Toolkit
- **Jenkins X:** Kubernetes-native CI/CD with GitOps
- **Rancher Fleet:** Multi-cluster GitOps at scale
- **Codefresh:** Enterprise platform with GitOps capabilities
- **Spinnaker:** Multi-cloud continuous delivery

Benefits



- **Technical Benefits**

- Increased deployment frequency and velocity
- Faster mean time to recovery (MTTR)
- Reduced configuration drift
- Automatic disaster recovery
- Self-healing infrastructure
- Declarative and reproducible deployments

- **Business Benefits**

- Improved productivity and developer experience
- Enhanced security and compliance
- Complete audit trail for regulatory requirements
- Reduced operational costs
- Better collaboration between teams
- Faster time to market

- **Operational Benefits**

- Simplified rollbacks (Git revert)
- Environment consistency across dev/staging/prod
- Infrastructure versioning and tracking
- Reduced human error
- Better disaster recovery capabilities

Security Advantages



- **Least Privilege Access:** No external credentials to production
- **Audit Trail:** Every change tracked in Git with author and timestamp
- **Code Review:** Changes go through pull request review process
- **Compliance:** Meets regulatory requirements for change management
- **Secret Management:** Integration with tools like Sealed Secrets, SOPS
- **Immutable Infrastructure:** No direct cluster access needed

Challenges & Considerations



■ Potential Challenges

- Initial learning curve and cultural shift
- Requires Git proficiency across teams
- Secret management complexity
- May need repository restructuring
- Requires discipline in maintaining Git hygiene
- Initial setup and tooling overhead
- Handling large-scale multi-cluster deployments

■ Mitigation Strategies

- Comprehensive training programs
- Start with pilot projects
- Use proper secret management tools
- Establish clear Git workflows and conventions

Repository Patterns



- **Common Structures**
 - **Monorepo:** Single repository for all applications and environments
 - **Repo per Application:** Separate repos for each application
 - **Repo per Environment:** Different repos for dev/staging/ production
 - **Hybrid Approach:** Combination based on organizational needs
- **Best Practices**
 - Separate application source from deployment configuration
 - Use separate repos for sensitive production configs
 - Maintain clear directory structure
 - Tag releases appropriately

Best Practices



- Keep application code and deployment configuration separate
- Use branches for different environments or feature testing
- Implement proper RBAC (Role-Based Access Control)
- Automate everything possible
- Use signed commits for enhanced security
- Implement progressive delivery (canary, blue-green)
- Monitor Git repository and cluster state continuously
- Document your GitOps workflows clearly
- Establish clear ownership and responsibilities
- Use policy-as-code tools (OPA, Kyverno)
- Regular backup of Git repositories

Real-World Use Cases



■ Practical Applications

- **Multi-environment Management:** Consistent deployments across dev/staging/prod
- **Multi-cluster Orchestration:** Managing hundreds of Kubernetes clusters
- **Disaster Recovery:** Rapid cluster rebuilding from Git
- **Compliance & Auditing:** Meeting regulatory requirements
- **Platform Engineering:** Building internal developer platforms
- **Edge Computing:** Managing distributed edge deployments

■ Industry Examples

- Financial services for regulatory compliance
- E-commerce for rapid feature deployment
- SaaS providers for multi-tenant environments
- Telecommunications for edge infrastructure



Future Trends

- Increased AI/ML integration for intelligent deployments
- Better secret management solutions
- Enhanced multi-cloud support
- Improved developer experience tools
- Integration with platform engineering initiatives
- Standardization through OpenGitOps project
- WebAssembly and edge computing integration



Common Misconceptions

- GitOps ≠ Just using Git for configuration
- Not limited to Kubernetes only
- Not a replacement for CI/CD, but a complementary approach
- Doesn't eliminate the need for testing
- Not automatically secure without proper implementation



GitHub Actions

Introduction



- GitHub Actions is a native CI/CD platform integrated into GitHub that allows you to automate workflows directly in your repository
- **Key Features:**
 - Event-driven automation
 - Matrix builds for testing across multiple environments
 - Reusable workflows and actions
 - Built-in secret management
 - Native GitHub integration
 - Free tier available (2,000 minutes/month for private repos)

GitHub Actions vs Other CI/CD Tools



- **Comparison with Jenkins**

- **GitHub Actions:** Cloud-based, no infrastructure management
- **Jenkins:** Self-hosted, more control but more maintenance

- **Comparison with GitLab CI**

- **GitHub Actions:** Marketplace with thousands of actions
- **GitLab CI:** Integrated with GitLab ecosystem

- **Comparison with CircleCI**

- **GitHub Actions:** Native GitHub integration
- **CircleCI:** Strong Docker support, paid tiers



Core Concepts - Workflows

- A workflow is an automated process that you define in your repository
- Workflows are made up of one or more jobs and can be scheduled or triggered by events
- A **workflow** is a YAML automation file stored in: `.github/workflows/*.yaml`
- **A workflow defines**
 - When to run (trigger/ event)
 - What to run (jobs & steps)
 - Where to run (runner type)
 - How to run (actions or scripts)

Core Concepts - Events



- Events are specific activities that trigger a workflow
- Events tell GitHub *when* to start a workflow.
- GitHub monitors your repo for events such as:
 - push
 - pull_request
 - schedule (cron)
 - workflow_dispatch (manual run)
 - release
 - issue_comment
 - label
- When an event occurs, GitHub's **Actions service**
 - detects the event
 - loads your workflow
 - assigns runners

Core Concept - Runners



- A **runner** is a machine responsible for running your jobs
- **GitHub provides two types**
 - **GitHub-Hosted Runners**
 - Managed by GitHub, automatically provisioned.
 - Available OS: Ubuntu (most popular), Windows, macOS
 - Features:
 - Auto-scaled
 - Pre-installed tools
 - Ephemeral (fresh every run)
 - **Self-Hosted Runners**
 - You bring your own runner on:
 - Your laptop
 - On-prem VM
 - AWS EC2
 - Kubernetes cluster
 - Raspberry Pi
 - Advantages:
 - More power
 - Cheaper for heavy workloads
 - Persistent cache/files



Coore Concept - Actions

- Actions are standalone commands or reusable units of code that you can combine into steps
- You can create your own actions or use actions from the GitHub community
- **There are three types**
 - **Actions from Marketplace**
 - uses: actions/checkout@v4
 - **Custom Actions**
 - You write your own action
 - Types of custom actions
 - JavaScript actions
 - Docker container actions
 - Composite actions
 - **Scripted steps**
 - Commands you directly run inside workflow
 - run: npm install

Architectural Principles



- **Event-driven:** Just like serverless — everything runs only on triggers
- **Ephemeral runners:** Ensures clean environment & security
- **Distributed execution:** Jobs can run in parallel across multiple runners
- **Marketplace extensibility:** Developers contribute reusable actions.
- **Secure by design**
 - Includes:
 - Secrets
 - Permissions
 - OIDC for cloud auth
 - Fine-grained access controls

When to Use GitHub Actions



- **Ideal Use Cases**

- Automated testing on every commit
- Building and publishing Docker images
- Deploying to cloud platforms
- Releasing packages to registries
- Running security scans
- Automating issue management
- Generating documentation
- Scheduled maintenance tasks

- **May Not Be Ideal For**

- Extremely long-running processes (>6 hours per job)
- Workflows requiring specific hardware
- Highly sensitive environments (consider self-hosted runners)
- Projects not on GitHub

Pricing and Limits



■ Free Tier

- Public repositories: Unlimited
- Private repositories: 2,000 minutes/month
- 500 MB artifact storage

■ Minute Multipliers

- Linux: 1x
- Windows: 2x
- macOS: 10x

■ Paid Plans

- GitHub Pro: \$4/month + usage
- GitHub Team: \$4/user/month + usage
- GitHub Enterprise: Custom pricing

■ Limits

- Workflow run time: 72 hours
- Job execution time: 6 hours
- API requests: 1,000 per hour
- Concurrent jobs: Varies by plan (20-180)
- Workflow queue: 500 across all workflows



Getting Started with Workflows

- Workflows are defined in YAML files stored in `.github/workflows/` directory
- **name:** Workflow name displayed in GitHub UI
- **on:** Event(s) that trigger the workflow
- **jobs:** One or more jobs to execute
- **runs-on:** Specifies the runner environment
- **steps:** Sequential tasks within a job

```
name: My First Workflow

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Run a one-line script
        run: echo "Hello, GitHub Actions!"
```

Workflow Run Lifecycle



- Event occurs (push, PR, schedule, etc.)
- GitHub creates workflow run
- Jobs are queued
- Runner picks up job
- Steps execute sequentially
- Job completes (success/failure)
- Workflow run completes

Jobs



```
jobs:
  job-name:
    name: Display Name
    runs-on: ubuntu-latest
    steps:
      - name: Step 1
        run: echo "Running step 1"
```

Basic Job Creation

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building..."

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Testing..."

  deploy:
    needs: [build, test]
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying..."
```

Job Dependencies

```
jobs:
  deploy:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to production"
```

Conditional Jobs

Steps



```
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '20'
```

Using Actions

```
steps:
  - name: Single line command
    run: echo "Hello"

  - name: Multi-line script
    run: |
      echo "Line 1"
      echo "Line 2"
      npm install
      npm test
```

Running Commands

```
steps:
  - name: Always run
    run: echo "This always runs"

  - name: Run on success
    if: success()
    run: echo "Previous steps succeeded"

  - name: Run on failure
    if: failure()
    run: echo "Something failed"

  - name: Run always
    if: always()
    run: echo "Runs regardless of previous step status"
```

Step Conditions

Environment Variables



```
env:
  GLOBAL_VAR: "global value"

jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo $GLOBAL_VAR
```

Workflow Level

```
jobs:
  job1:
    runs-on: ubuntu-latest
    env:
      JOB_VAR: "job value"
    steps:
      - run: echo $JOB_VAR
```

Job Level

```
steps:
  - name: Use env var
    env:
      STEP_VAR: "step value"
    run: echo $STEP_VAR
```

Step Level

GITHUB_WORKFLOW: Workflow name
GITHUB_RUN_ID: Unique run identifier
GITHUB_RUN_NUMBER: Sequential run number
GITHUB_ACTOR: User who triggered workflow
GITHUB_REPOSITORY: Owner/repo name
GITHUB_SHA: Commit SHA
GITHUB_REF: Branch or tag ref

Actions



■ Core GitHub Actions

- **actions/checkout**: Checks out your repository into the runner
- **actions/setup-node**: Installs and configures Node.js
- **actions/setup-python**: Installs Python and manages versions
- **actions/setup-java**: Installs Java JDK (Temurin, Zulu, etc.)
- **actions/setup-go**: Installs Go toolchain
- **actions/setup-dotnet**: Installs .NET SDK
- **actions/setup-ruby**: Installs Ruby and bundler
- **actions/cache**: Caches build dependencies (npm, pip, gradle, etc.)

■ Docker-Related Official Actions

- **docker/login-action**: Login to Docker Hub or any registry
- **docker/build-push-action**: Build and push Docker images
- **docker/setup-buildx-action**: Set up Buildx for multi-arch builds
- **docker/setup-qemu-action**: Enable QEMU for multi-architecture builds



Advantages of GitHub Actions

- Deep Integration with GitHub
- Easy to Use & Quick Setup
- **GitHub-Hosted Runners (Free VMs)**
- Great for Open-Source
- Rich Ecosystem (Marketplace)
- Supports Complex CI/CD Pipelines
- Supports Self-Hosted Runners
- Strong Security Features



Disadvantages of GitHub Actions

- Limited Free Minutes for Private Repos
- Slower for Very Large Pipelines
- Harder Debugging Compared to Jenkins
- Vendor Lock-In
- **Marketplace Actions Have Trust Risk**
- Workflow Limits
- No Full CD Dashboard Like Jenkins
- Secrets Are Repository-Level Only